BEGINNER'S STUDY GUIDE

Practical Packet Analysis

Learn networking by reading the traffic itself
A plain-language guide to the book by Chris Sanders

English edition

Companion summary of “Practical Packet Analysis: Using Wireshark to Solve Real-World Network Problems” (No Starch Press, 2007).
Written in original words for learning. It does not replace the book.

How to use this guide

Read it in order, once, then come back to the cheat sheets at the end.

The book teaches networking through **packet analysis**: capturing the small chunks of data (packets) that travel across a network and reading them. When you can read packets, nothing on the network is hidden from you, which makes it the best way for a beginner to *really* understand how networks work.

- Every chapter below has the **big idea**, the **key concepts**, and things to **try**.
- The book comes with sample capture files (`.pcap`) at `nostarch.com/packet.htm`. Download them and open them in Wireshark while you read: this is the fastest way to learn.
- Chapters marked ★ are the most important for a beginner. If you are short on time, start with them.

> **CAREFUL — Golden rule**
> Only capture traffic on networks you own or are explicitly authorised to analyse. Sniffing other people's traffic without permission can be illegal and is unethical.

Roadmap of the book

Ch.	Title	In one sentence
1 ★	Packet Analysis and Network Basics	What packets are, the 7 OSI layers, and how hubs, switches and routers behave.
2	Tapping into the Wire	Where to plug in your sniffer so you actually see the traffic you want.
3 ★	Introduction to Wireshark	Install Wireshark, make your first capture, understand the 3-pane window.
4 ★	Working with Captured Packets	Find, mark, save, merge, time-read and filter packets.
5	Advanced Wireshark Features	Name resolution, Decode As, Follow TCP Stream, statistics windows.
6 ★	Common Protocols	How ARP, DHCP, TCP/IP, HTTP, DNS, FTP, Telnet and ICMP look when they work.
7	Basic Case Scenarios	Lost connections, unreachable hosts, fragments, and the first real troubleshooting cases.
8	Fighting a Slow Network	Seven cases showing that “the network is slow” is often not the network's fault.
9	Security-based Analysis	Spotting scans, break-ins, worms and hidden communication.
10	Sniffing into Thin Air	Capturing Wi-Fi traffic.
11	Further Reading	Extra tools and resources.

> **BEGINNER TIP — Study order for absolute beginners**
> Chapter 1 → 3 → 4 → 6 first (about 70% of the foundations). Then 2 and 5. Finally the scenario chapters 7–9, which are where everything clicks together.

Chapter 1 – Packet Analysis and Network Basics ★

Goal: understand what you are looking at before you look at it.

1.1 What is packet analysis?

Packet analysis (also called packet sniffing or protocol analysis) is capturing live data as it crosses a network and interpreting it. A **packet sniffer** is the tool that does it. It helps you learn who is on the network, what is using bandwidth, when the busy hours are, whether something malicious is going on, and which applications are bloated or insecure.

Popular sniffers: **tcpdump** (command line), **OmniPeek** and **Wireshark** (graphical). When choosing one, compare: supported protocols, user-friendliness, cost, support/community, and operating-system support.

1.2 How a sniffer works: 3 steps

Step	What happens
1. Collection	The sniffer puts the network card in **promiscuous mode** (it listens to all traffic on its segment) and grabs raw binary data from the wire.
2. Conversion	The raw binary is converted into a human-readable form. Simple tools stop here.
3. Analysis	The tool recognises the protocol and interprets its fields, comparing multiple packets to find patterns.

1.3 Protocols and the OSI model

A **protocol** is a common language computers use to talk. A **protocol stack** is a group of protocols working together. Protocols deal with problems such as: flow control, acknowledgements, error detection and correction, segmentation, encryption and compression.

The **OSI model** splits communication into 7 layers, each with a specific job. Data goes *down* the layers on the sender and *up* the layers on the receiver.

#	Layer	Job	Examples
7	Application	The programs people use to reach network resources.	HTTP, SMTP, FTP, Telnet
6	Presentation	Formats data so the application can read it; some encryption.	ASCII, JPEG, MPEG
5	Session	Opens, manages and closes the dialogue between two computers.	NetBIOS, SAP
4	Transport	Reliable delivery: flow control, splitting data, error control.	TCP, UDP
3	Network	Logical addressing (IP) and routing between networks. **Routers** live here.	IP, ICMP, ARP*
2	Data Link	Physical addressing (MAC) on one network. **Switches** live here.	Ethernet
1	Physical	Cables, signals, hubs, network adapters.	Cables, hubs

**The book classes ARP at Layer 3 but notes it serves both Layers 2 and 3.*

> **ANALOGY — Sending a letter**

You write a letter (application), put it in an envelope (transport adds a header), add the street address (network), hand it to the postman who uses local routes (data link) and trucks and roads carry it (physical). The receiver opens each wrapper in reverse order.

BEGINNER TIP — Memory trick
Top to bottom: **A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing (Application, Presentation, Session, Transport, Network, Data link, Physical).

1.4 Encapsulation and PDUs

Each layer adds its own **header** (sometimes a footer) to the data it receives from above. This is **encapsulation**. The result, data plus all added headers, is a **Protocol Data Unit (PDU)**. In this book, a **packet** means the complete PDU. The receiver removes the headers layer by layer until only the original data remains.

1.5 Network hardware: hub, switch, router

Device	Layer	How it handles traffic	Why it matters for sniffing
Hub	1	Repeats every packet to *all* other ports. Half-duplex. Causes collisions.	Easiest: plug in and see everything.
Switch	2	Learns each device's MAC address in a **CAM table** and sends packets only to the right port. Full-duplex; many conversations at once.	You only see your own traffic unless you use a trick (Ch. 2).
Router	3	Forwards packets *between* networks using IP addresses (routing). A Layer 3 switch can do both jobs.	Traffic seen on one side may not be visible on the other side.

ANALOGY — A neighbourhood
A switch is the street: everybody on it can reach each other. A router is the street sign that lets you reach a house on another street. A hub is shouting down the street so everyone hears you.

1.6 Types of traffic and broadcast domains

- **Unicast**: one sender to one receiver.
- **Multicast**: one sender to a chosen group of receivers (efficient on bandwidth).
- **Broadcast**: one sender to everyone on the segment.
- **Broadcast domain**: the area where a broadcast can reach. It ends at a router.

REMEMBER — Chapter 1 in three lines
1) Every network problem exists at the packet level. 2) The 7 OSI layers add headers on the way down and remove them on the way up. 3) Hubs repeat to all, switches target one port, routers connect networks.

Chapter 2 – Tapping into the Wire

Goal: put the sniffer where the interesting traffic actually passes.

Placing a sniffer is sometimes harder than analysing the packets. The right method depends on the hardware between the computers.

Promiscuous mode

A normal network card ignores packets not addressed to it. In **promiscuous mode** it passes everything it hears to the CPU so the sniffer can see it. It usually needs administrator rights. If you cannot get them, you probably should not be sniffing on that network.

Sniffing methods by environment

Environment	What you can see	Method
Hub	Everything (limitless “visibility window”).	Plug into any free port.
Switch	Only your own traffic by default.	**Port mirroring** (SPAN), **hubbing out**, or **ARP cache poisoning**.
Routed network	Only your segment.	Same techniques, plus capture on **several sides** of the router.

Port mirroring

You configure a managed switch to copy all traffic of one (or more) port(s) to the port where your sniffer is connected. Commands differ by vendor (for example Cisco uses `set span <source> <destination>`). Beware of mirroring too many busy ports into one: the destination port can be overloaded and packets will be dropped.

Hubbing out

Unplug the target from the switch, plug it into a **hub**, plug your analyser into the same hub, and connect the hub to the switch. Now the target and your sniffer share a broadcast domain. It lowers the target to half-duplex, so it is not elegant, but it works when mirroring is unavailable. Test that the “hub” is a real hub (some are sold as hubs but behave like switches): connect two computers and check that one can see the other's traffic.

ARP cache poisoning

ARP links IP addresses to MAC addresses. In **ARP cache poisoning** (ARP spoofing), fake ARP messages make the target send its traffic through your machine, which becomes the “man in the middle”. Tools like Cain & Abel automate it. Attackers use it, but it is also a legitimate capture method on your own network.

> **CAREFUL — Do not overload the middle man**
> Your analyser forwards *all* traffic of the target. Poisoning a busy server through a slower link of yours can create a denial-of-service effect and ruin your analysis.

Routed networks and network maps

When a problem crosses several routers, capture on each side. Example: network D cannot talk to network A. Sniffing on D shows data leaving with no replies; sniffing the next network up shows the router dropping it, which points to a router configuration problem. Keep a **network map** (diagram of all devices and links) nearby: it tells you where to place the sniffer.

Chapter 3 – Introduction to Wireshark ★

Goal: install it, capture, and know what each part of the window shows.

Wireshark began as **Ethereal** (1998, Gerald Combs) and was renamed in 2006. It is free and open source, supports hundreds of protocols (over 850 at the time of the book), has a friendly graphical interface, active community support and runs on Windows, macOS and Linux.

Installing

- **Requirements:** a modest computer, a network card that supports promiscuous mode, and a capture driver. On Windows it is **WinPcap**, bundled with the Wireshark installer (tick "Install WinPcap").
- **Windows:** download from wireshark.org, run the installer and accept defaults.
- **Linux:** use your package manager (e.g. `apt-get install wireshark` on Debian/Ubuntu, or an RPM).

> **BEGINNER TIP — A note on dates**
> The book is from 2007. Menus in newer Wireshark versions have moved a little (for example Expert Infos is now under Analyze → Expert Information, and modern Windows uses Npcap). The ideas are unchanged.

Your first capture

> **BIG IDEA — There is always something to analyse**
> You do not need a problem to capture. Analysts spend most of their time looking at healthy traffic, because you need a **baseline** of "normal" to recognise "abnormal".

- Open Wireshark → **Capture → Interfaces** and choose the active interface → **Capture**.
- Let it run about a minute (browse a website, ping something).
- Click **Stop**. The main window fills with packets.

The main window: 3 panes

Pane	Shows	Use it to…
Packet List (top)	One row per packet: number, time, source, destination, protocol, info.	Get the overview and select a packet.
Packet Details (middle)	The selected packet broken into layers you can expand.	Read each protocol's fields (IP, TCP, HTTP…).
Packet Bytes (bottom)	The raw bytes (hex + text) of the packet.	Inspect raw data; click a field above to highlight its bytes.

The panes depend on each other: select in the list, expand in the details, see the bytes below.

Preferences and colours

Edit → Preferences has five areas: user interface, capture, printing, name resolution, protocols (leave protocol settings alone unless you have a reason). Packets are **colour-coded by protocol** (for example DNS blue, HTTP green). You can change them in **View → Coloring Rules**; e.g. colour DHCP bright yellow when hunting a rogue DHCP server.

> **TRY IT — Exercise**
> Capture 1 minute of normal browsing. In the Packet Details pane, expand Ethernet, IP and TCP for one packet and find the source IP, destination IP and port numbers.

Chapter 4 – Working with Captured Packets ★

Goal: handle big captures without drowning.

Finding and marking

- **Find Packet** (Ctrl+F): search by display filter, hex value or text string. Next match: Ctrl+N, previous: Ctrl+B.
- **Mark** a packet with Ctrl+M (it turns black with white text). You can later save only marked packets.

Saving, exporting, merging, printing

- **File → Save As**: default format is `.pcap`. You can save only a range, the marked packets, or the displayed (filtered) packets, which is great for trimming huge files.
- **Export** to plain text, PostScript, CSV or XML.
- **Merge** two captures: prepend, append, or chronologically.
- **Print** selected ranges or panes.

Time matters

Every packet has a timestamp. In **View → Time Display Format** you can show absolute time, time since the previous packet, or time since the start of the capture, with chosen precision. **Time reference** (Ctrl+T) makes one packet "time zero" (shown as *REF*), useful when a capture holds several requests. It only makes sense with the "seconds since beginning of capture" format.

Capture filters vs display filters

	Capture filter	Display filter
When	*Before/while* capturing.	*After* capturing, on the loaded file.
Effect	Unwanted packets are never stored.	Packets are hidden, not deleted: clear the filter to see all again.
Typical use	Busy server: only capture `port 262`.	Hide noise: `!arp`. Much more common.
Where	Capture → Options.	Filter box above the Packet List.

Beginners can build filters with the **Filter Expression** dialog (pick protocol field, relation, value). With practice, typing them is faster.

Display filter syntax

Operator	Meaning	Operator	Meaning
`==`	equal	`and`	both true
`!=`	not equal	`or`	either true
`>` / `<`	greater / less	`xor`	exactly one true
`>=` / `<=`	greater-or-equal / less-or-equal	`not` or `!`	negation

Filter	Result
`tcp`	Only TCP packets.
`!icmp`	Everything except ICMP (ping noise).
`ip.addr==192.168.0.1`	Packets to or from that IP.
`ip.addr==192.168.0.1 or ip.addr==192.168.0.2`	Either of the two IPs.
`ip.dst==192.168.0.1`	Packets going to that IP.

Filter	Result
`frame.pkt_len <= 128`	Short packets (128 bytes or less).
`!dns`	Everything except DNS.
`not broadcast and not multicast`	Unicast only (capture-filter syntax).

Good filters can be saved (Capture → Capture Filters / Analyze → Display Filters) and reused.

> **REMEMBER — Chapter 4 in one line**
> Capture a little more than you need, then use **display filters** (they are reversible) to narrow down, and use time formats to measure delays.

Chapter 5 – Advanced Wireshark Features

Goal: make cryptic data readable and see the big picture.

Name resolution

Converts hard-to-read addresses into friendly names. Three kinds:

Type	Converts	Example
MAC	Layer 2 address → name; falls back to the manufacturer from the first 3 bytes.	00:09:5B:01:02:03 → Netgear_01:02:03
Network	IP address → DNS name.	192.168.1.50 → MarketingPC1
Transport	Port number → service name.	80 → http

Enable it in Capture Options. **Drawbacks:** it can fail, it is redone each time you open the file, DNS lookups add extra packets to the capture, and it costs processing power on big files.

Protocol dissection and “Decode As”

A **dissector** is the translator that turns raw bytes into a readable protocol. Wireshark guesses the dissector, often from the port number. If a service runs on a non-standard port (the book's example: FTP on port 137, normally NetBIOS), Wireshark can guess wrong. Fix it with **right-click → Decode As** (a “forced decode”). It is not saved in the capture file, so redo it each time.

Follow TCP Stream

Reassembles the small pieces of a conversation into what the application saw: the full chat, web page or login exchange, in readable form. Right-click a packet → **Follow TCP Stream**. In the book it reveals an entire instant-messenger chat.

> **CAREFUL — Lesson in security**
> Unencrypted protocols (FTP, Telnet, old chat apps) expose passwords and messages to anyone who can sniff. This is why encryption matters.

Statistics windows

Window	Menu	What it tells you
Protocol Hierarchy	Statistics → Protocol Hierarchy	Share of each protocol in the capture. Great for benchmarking: if ARP is normally 10% and today is 50%, investigate.
Endpoints	Statistics → Endpoints	Each address that sent/received data and how many packets/bytes.
Conversations	Statistics → Conversations	Pairs of endpoints talking, with packets/bytes each way. Right-click to create filters.
IO Graphs	Statistics → IO Graphs	Traffic over time, to spot spikes and patterns.

An **endpoint** is one side of communication (an IP or a MAC). A **conversation** is the traffic between two endpoints.

> **ANALOGY — Endpoints and conversations**
> Jim and Sally are endpoints; what they say to each other is a conversation. For computers, a TCP conversation is “SYN, SYN/ACK, ACK…”.

Chapter 6 – Common Protocols ★

Goal: learn what "working normally" looks like so you can spot what is broken.

Each protocol has an official document called an **RFC** (rfc-editor.org). The book shows each one in a small capture file.

ARP – "Who has this IP?" (RFC 826)

Translates an IP address (Layer 3) into a MAC address (Layer 2). The sender broadcasts to `ff:ff:ff:ff:ff:ff`: "Who has 192.168.0.1?" Only the owner answers with its MAC. Everyone else ignores it.

> **ANALOGY — Phone book**
> You don't know which Smith to call, so you call every Smith until the right one answers.

DHCP – automatic network settings (RFC 2131)

Gives clients an IP address and other settings. Four steps, often remembered as **DORA**:

Step	Packet	Meaning
1	**D**iscover (broadcast)	Client: "Is there any DHCP server?"
2	**O**ffer	Server: "Here is an address you can use."
3	**R**equest (broadcast)	Client: "I accept that offer." Tells other servers to stand down.
4	**A**CK	Server: "Confirmed." The transaction ends.

Each exchange has a **Transaction ID** so the server and you can keep clients apart.

TCP/IP and HTTP

IP (RFC 791, Layer 3) addresses and delivers packets but is connectionless. **TCP** (RFC 793, Layer 4) adds reliable, two-way communication. **HTTP** (RFC 2616) carries web pages on top of TCP.

The TCP three-way handshake

```
Client                          Server
  |---- SYN  (seq = x) --------->|   "Let's talk, my number is x"
  |<--- SYN/ACK (ack = x+1) -----|   "OK, got it; my number is y"
  |---- ACK  (ack = y+1) ------->|   "Got yours too. Ready."
```

- **SYN**: starts synchronisation; carries a 32-bit sequence number.
- **SYN/ACK**: server acknowledges (your number + 1) and sends its own.
- **ACK**: client confirms. Data can now flow.
- Wireshark shows **relative** sequence numbers by default (starting at 0) to make them easier to follow.
- After the handshake, sequence numbers grow by the amount of data sent.

HTTP request and closing the session

The client sends an HTTP **GET** for a page (e.g. `/download.html` on a host). The server replies **200 OK** and sends the data over TCP. To close, each side sends **FIN/ACK** and the other answers with **ACK**, so the end takes two small exchanges, one per direction.

DNS – names to IP addresses (RFC 1034)

Usually only two packets: a **query** (“What is the IP of www.example.com?”) and a **response** with the address. Without it Layer 3 could not find the computer. One web page often triggers many DNS queries.

FTP – file transfer (RFC 959)

Layer 7, ports 20 and 21. After a TCP handshake the server greets, the client logs in (user name and password, **in clear text**), then sends commands:

Command	Meaning
`CWD`	Change working directory.
`SIZE`	Ask the size of a file (the server replies with bytes).
`RETR`	Retrieve (download) a file. Packets marked FTP-DATA carry the file.

Telnet, MSN Messenger and ICMP

- **Telnet**: remote command line; also unencrypted, so everything typed can be read.
- **MSN Messenger** (an old instant messenger): command packets like USR, CAL, JOI, then MSG packets with the messages. Follow TCP Stream shows the full chat.
- **ICMP** (RFC 792): the “utility protocol” used by **ping**. Echo request is **type 8**; echo reply is **type 0**. A Windows ping sends four requests.

> **REMEMBER — Typical normal sequence of a web visit**
> DNS lookup → TCP handshake → HTTP GET → 200 OK + data → FIN/ACK on both sides. If you can recite this, you can spot what goes wrong.

Chapter 7 – Basic Case Scenarios

Goal: practise the method on small problems.

> **BIG IDEA — The troubleshooting method used in every case**
> 1) **What we know**: write down the facts. 2) **Tapping into the wire**: decide what to capture and where. 3) **Analysis**: read the packets. 4) **Summary**: state the cause and the lesson.

A lost TCP connection

The capture shows normal ACKs, then **TCP retransmissions**. TCP sends a packet; if no reply arrives it waits, resends, and **doubles the waiting time** each time. A run of retransmissions with no answer is what a lost connection looks like at the packet level.

Unreachable destinations and ICMP

A failed ping often returns a **Destination unreachable** message (**ICMP type 3**) instead of an echo reply. The **code** says why. The key clue: the reply comes from a device that is *not* the host you pinged, so the request never arrived.

ICMP type 3 code	Meaning
1	Host unreachable: reached the upstream router/switch but not the host (you often also see an unanswered ARP broadcast).
3	Port unreachable: the host is there but nothing accepts connections on that port.

Note: the book's text is slightly inconsistent about type/code numbers; the values above follow the official ICMP standard.

Fragmented packets

Only so much data fits in one packet, so IP can split data into **fragments**. In the IP details, the **More Fragments** flag is 1 on every fragment except the last (0), and the fragment offset keeps them in order.

Case: No connectivity (Barry and Beth)

Two identical new PCs; Barry's reaches the Internet, Beth's does not. Method: capture on both and **compare the working one with the broken one** to find the first packet where they differ. Cause: a typo in Beth's default gateway (192.168.0.11 instead of 192.168.0.10). Lesson: most errors are misconfigurations.

Other cases in the chapter

Case	Symptom	How it was approached / lesson
The ghost in Internet Explorer	Browser home page keeps changing to weather sites.	Capture from boot time using hubbing out, since the symptom reappears at start-up.
Inbound FTP	New FTP server unreachable from outside.	Capture on **both** client and server to learn which side the problem is on.
It's not my fault!	A user's form submission returns HTTP 403 Forbidden.	Follow TCP Stream shows the server itself refusing: proof for the sceptical user and the boss.
An evil program	Hijacked home page, pop-ups, slow PC (spyware).	Capture at boot with a filter for that PC. Found the spyware arriving through a background RPC service; learning its ports lets you block it on the firewall.

REMEMBER — Lesson
Even when the fix looks obvious, find out *exactly why* it happened. It lets you prevent it for everyone else.

Chapter 8 – Fighting a Slow Network

Goal: find out whether the network is really slow, and what is truly slowing things.

BIG IDEA — Main lesson of this chapter
"The network is slow" is the most common complaint, and usually the problem is a computer, an application or a user, not the network.

Anatomy of a slow download

Use **Analyze → Expert Infos** (set the severity filter to Error+Warn+Note) to jump straight to abnormal packets. In the example you see many **TCP Window Update** packets and **Previous segment lost** warnings. The **TCP receive window** controls how fast data can flow.

ANALOGY — Water fountain
Someone presses the button for you. Too hard and you can't swallow everything (window shrinks), too soft and you drink little (window grows). Window updates are you asking them to press harder or softer.

The cases

Case	Clue in the packets	Cause
A slow route	**traceroute** sends packets with increasing **TTL**; each router replies "TTL expired". The first hop waited about 3 seconds before answering.	The internal router was the bottleneck: a router configuration issue.
Double vision	Every packet appears twice. Typical causes: routing inconsistency or bad port mirroring.	A **wrong subnet mask** on the PC made every packet bounce back, doubling the traffic.
Did that server flash me?	Browser just loads and times out; packets show the connection being refused.	A pop-up blocker blocked the Flash window the site opens for its content.
A torrential downfall	Everyone is slow; edge router CPU is high; huge traffic from one workstation.	A user ran BitTorrent with unlimited upload and download.
POP goes the email server	Huge number of POP packets. Set time format to "seconds since beginning of capture" to compute the rate.	Spam with large attachments overloaded the mail server.
Here's something Gnu	Conversations window: 81 IP and 243 TCP conversations from one workstation, mostly unanswered SYNs or RST replies.	An unauthorised peer-to-peer (Gnutella) file-sharing program.

BEGINNER TIP — Useful tools in this chapter
Expert Infos (finds anomalies), Conversations (counts and sizes), time display formats (rates), traceroute (finds the slow hop), IO Graphs (spikes).

Chapter 9 – Security-based Analysis

Goal: recognise attackers' footprints. Learn to defend by understanding offence.

Topic	What it is	What you see in packets
OS fingerprinting	An attacker sends varied probes and deduces the target's operating system from its replies, to pick matching exploits.	A series of unusual probe packets from one source to one target.
Port scan	Trying many ports (21 FTP, 80 HTTP…) to find “open doors” into a network.	One host sending connection attempts to many ports on another host.
Mystery traffic to a printer	Unexpected data (SPOOLS) hitting a printer.	Wireshark quickly identifies the source client, probably compromised.
FTP break-in	Unexplained after-hours traffic on an old FTP server with no logging.	Capture shows who connects and when; the packet capture replaces the missing logs.
Blaster worm	A PC reboots every 60 seconds even with antivirus installed.	Identify the worm's traffic, then search the symptoms online to learn the fix.
Covert channel (Loki)	Two staff exchange hidden messages inside ICMP packets.	Look in the **Packet Bytes** pane: hidden data can also appear in TCP headers or ARP.

BEGINNER TIP — Why it matters
Knowing the ports and services an attack uses lets you block them at the firewall. Always keep the Packet Bytes pane in mind; sometimes it is the only place a secret appears.

CAREFUL — Defence only
Use these techniques to protect your own networks. Scanning or attacking systems you do not own is illegal.

Chapter 10 – Sniffing into Thin Air (Wireless)

Goal: capture and read Wi-Fi traffic.

- **Managed mode**: your card talks to an access point which manages the communication; you see only your own traffic.
- **Monitor mode**: the card listens passively to everything on a channel. Needed for real wireless sniffing.
- **Windows**: most drivers do not allow monitor mode, so the book uses an **AirPcap** USB adapter (choose channel, include FCS, enter WEP keys).
- **Linux**: enable monitor mode with the wireless tools (`iwconfig`) as root, then start Wireshark.
- Wireless packets add an **802.11 header** with extra fields; the **DS status** flags tell the direction (access point → client or client → access point).

Frame type / subtype	Display filter
Management frames	`wlan.fc.type eq 0`
Control frames	`wlan.fc.type eq 1`
Data frames	`wlan.fc.type eq 2`
Beacon	`wlan.fc.type_subtype eq 8`
Disassociate	`wlan.fc.type_subtype eq 10`
Authentication	`wlan.fc.type_subtype eq 11`

Frame type / subtype	Display filter
Deauthentication	`wlan.fc.type_subtype eq 12`

BEGINNER TIP — Modern note
The book's examples use WEP encryption, which is obsolete and insecure today. Modern networks use WPA2/WPA3, and decrypting them in Wireshark needs different settings.

Chapter 11 – Further Reading

- **Cain & Abel**: ARP poisoning, password sniffing and recovery, VoIP recording, network discovery.
- **PingPlotter**: graphs ping/traceroute results over time to show connectivity trends.
- **AirPcap**: USB adapter for Wi-Fi capture on Windows.
- **RFC Editor** (rfc-editor.org): official protocol documents.
- **Wireshark website**: documentation, wiki, FAQs and mailing lists.

Cheat sheets

Print these two pages.

A. Troubleshooting method

- **1. Define the symptom**: who, what, when? One user or everyone?
- **2. List what you know** and what you don't.
- **3. Choose where to capture** (client, server, both, each side of a router) and how (hub, port mirror, ARP poisoning, or install Wireshark locally).
- **4. Capture** at the right moment (for example at boot).
- **5. Compare** with a working machine or a baseline.
- **6. Narrow with filters**, Expert Infos, Conversations, Follow TCP Stream.
- **7. State the cause**, fix it, and write down the lesson.

B. Quick glossary

Term	Meaning
Packet / PDU	Data plus all headers added by the layers.
Sniffer	Tool that captures and reads packets.
Promiscuous mode	NIC mode that accepts all traffic, not only its own.
MAC address	Layer 2 hardware address of a network card.
IP address	Layer 3 logical address.
Port	Number identifying a service on a host (80 = HTTP, 21 = FTP).
Broadcast domain	Area a broadcast can reach; ends at a router.
CAM table	Switch's table of MAC addresses and ports.
Baseline	What normal traffic looks like on your network.
Dissector	Wireshark module that decodes a protocol.
Endpoint / Conversation	One side of communication / traffic between two endpoints.
TTL	Maximum router hops a packet may travel; used by traceroute.
RFC	Official document defining an Internet protocol.

C. Common ports and TCP/ICMP signals

Item	Meaning
Ports 20/21, 80	FTP data/control, HTTP (the book's examples).
Ports 23, 25, 53, 110	Telnet, SMTP (mail sending), DNS, POP3 (mail retrieval): standard assignments.
SYN → SYN/ACK → ACK	TCP connection opens.
FIN/ACK each way	TCP connection closes politely.
RST	Connection refused or aborted.
Repeated retransmissions	Lost packets or a dead path.
ICMP type 8 / 0	Ping request / ping reply.
ICMP type 3	Destination unreachable (code tells why).

Self-test

Answer on paper first, then read the answers.

1. Name the 7 OSI layers from top to bottom.
Answer: Application, Presentation, Session, Transport, Network, Data Link, Physical.

2. What is the difference between a hub and a switch?
Answer: A hub repeats every packet to all ports; a switch uses its CAM table of MAC addresses to send packets only to the right port.

3. Why can't you simply plug a sniffer into a switch and see everything?
Answer: The switch sends traffic only to its destination port. You need port mirroring, hubbing out or ARP cache poisoning.

4. What is promiscuous mode?
Answer: A network-card mode that passes all received traffic to the sniffer instead of ignoring packets not addressed to it.

5. Which Wireshark pane shows the raw bytes?
Answer: The Packet Bytes pane (bottom).

6. Capture filter or display filter: which one can be undone without recapturing?
Answer: The display filter; it only hides packets.

7. Write a display filter showing all traffic except DNS.
Answer: `!dns` (or `not dns`).

8. List the four DHCP steps.
Answer: Discover, Offer, Request, ACK.

9. Describe the TCP three-way handshake.
Answer: SYN from client; SYN/ACK from server; ACK from client.

10. What does a series of TCP retransmissions with growing delays suggest?
Answer: Packets are not being acknowledged: lost connectivity or a dead path.

11. What did the “double vision” case turn out to be?
Answer: A wrong subnet mask on the PC, causing every packet to bounce back and appear twice.

12. What is Follow TCP Stream used for?
Answer: Reassembling a conversation to read, in clear text, what the application exchanged.

13. Why is a baseline useful?
Answer: To recognise abnormal traffic you must know what normal traffic looks like.

Four-week study plan

Week	Read	Practise
1	Chapters 1 and 3	Install Wireshark, capture a minute of browsing, identify the 3 panes and 5 protocols in it.
2	Chapters 4 and 6	Open arp, dhcp, http, dns, ftp and icmp sample captures. Write each flow in your own words. Build 10 display filters.
3	Chapters 2 and 5	Draw your home network map. Try Follow TCP Stream, Conversations, Protocol Hierarchy.
4	Chapters 7 – 10	Solve each scenario *before* reading the solution. Then explore chapter 11 tools.

REMEMBER — Final advice

> Packet analysis is a skill built by repetition. Open a capture every day, even a healthy one. The more normal traffic you have seen, the faster the abnormal one stands out.